



Departamento de Informática e Matemática Aplicada – DIMAp

Spring Discovery Service and Spring Cloud Loadbalancer

Prof. Nélio Cacho

Universidade Federal do Rio Grande do Norte
neliocacho@dimap.ufrn.br

Hora de silenciar o celular...

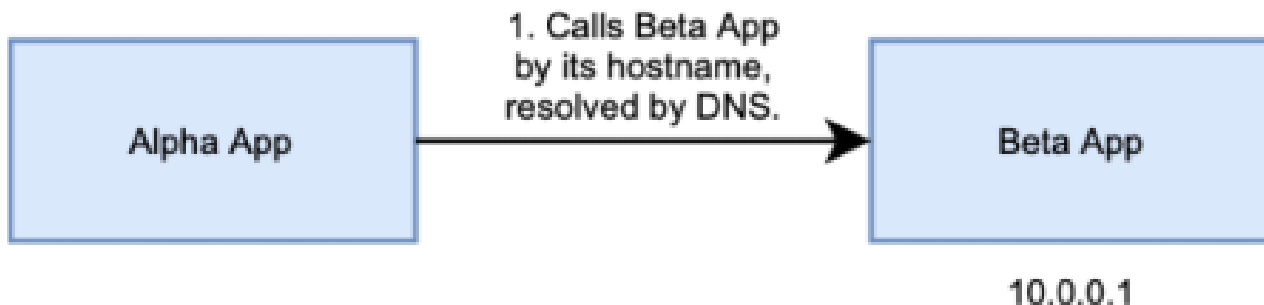


- Manter o telefone celular sempre desligado/silencioso quando estiver em sala de aula;
- Nunca atender o celular na sala de aula.

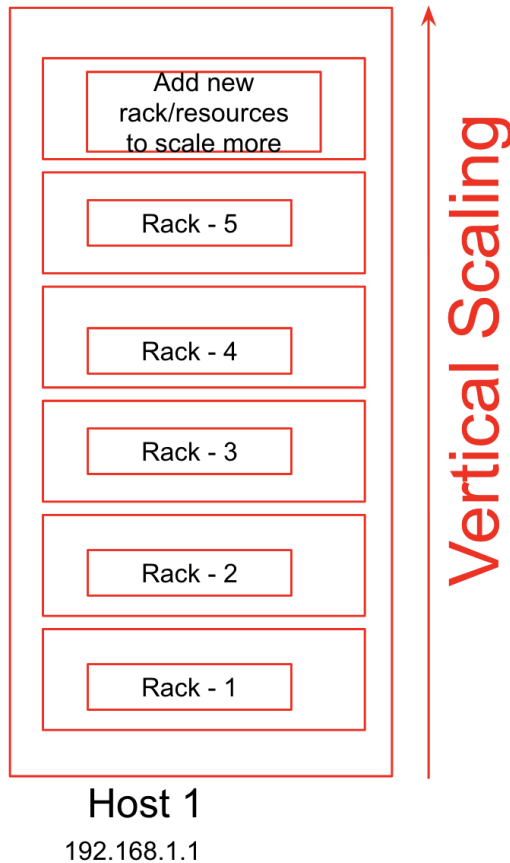
Entendendo o Serviço de Descoberta e LB

- Quando um serviço precisa se comunicar com outro, é necessário fornecer informações sobre onde encontrá-lo. Por exemplo, pode ser um endereço IP ou um nome DNS.

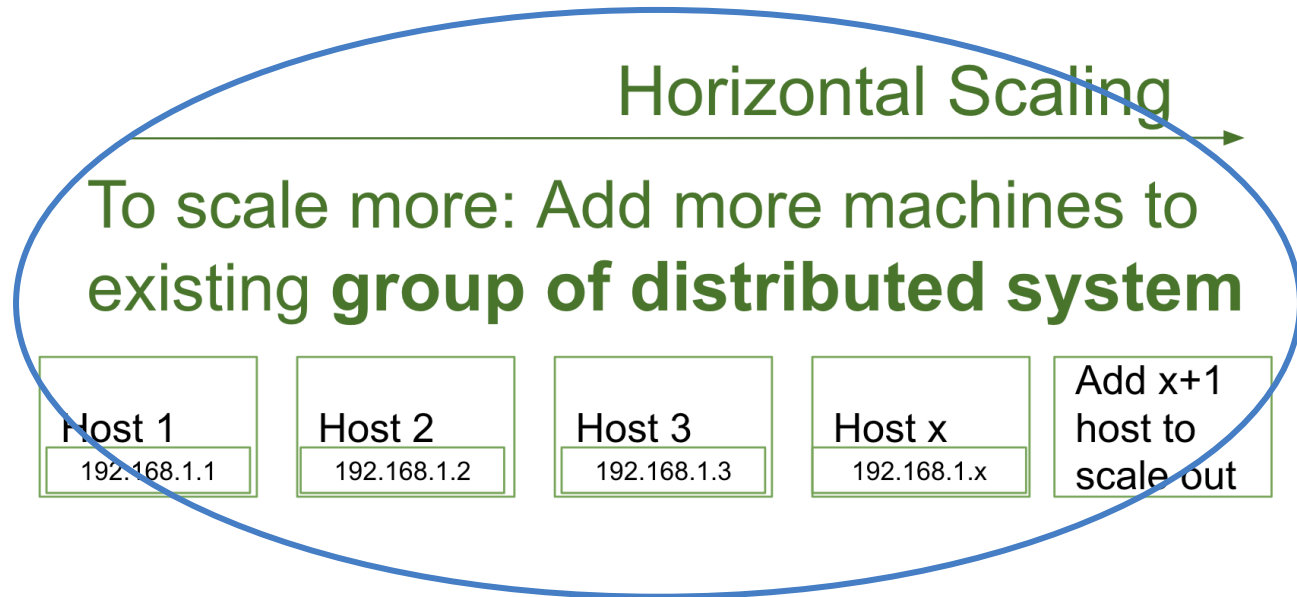
Inter-process communication - No service discovery or load balancing



Escalabilidade horizontal e escalabilidade vertical



To scale more, Add more RAM, CPU, Memory to the **one existing machine**

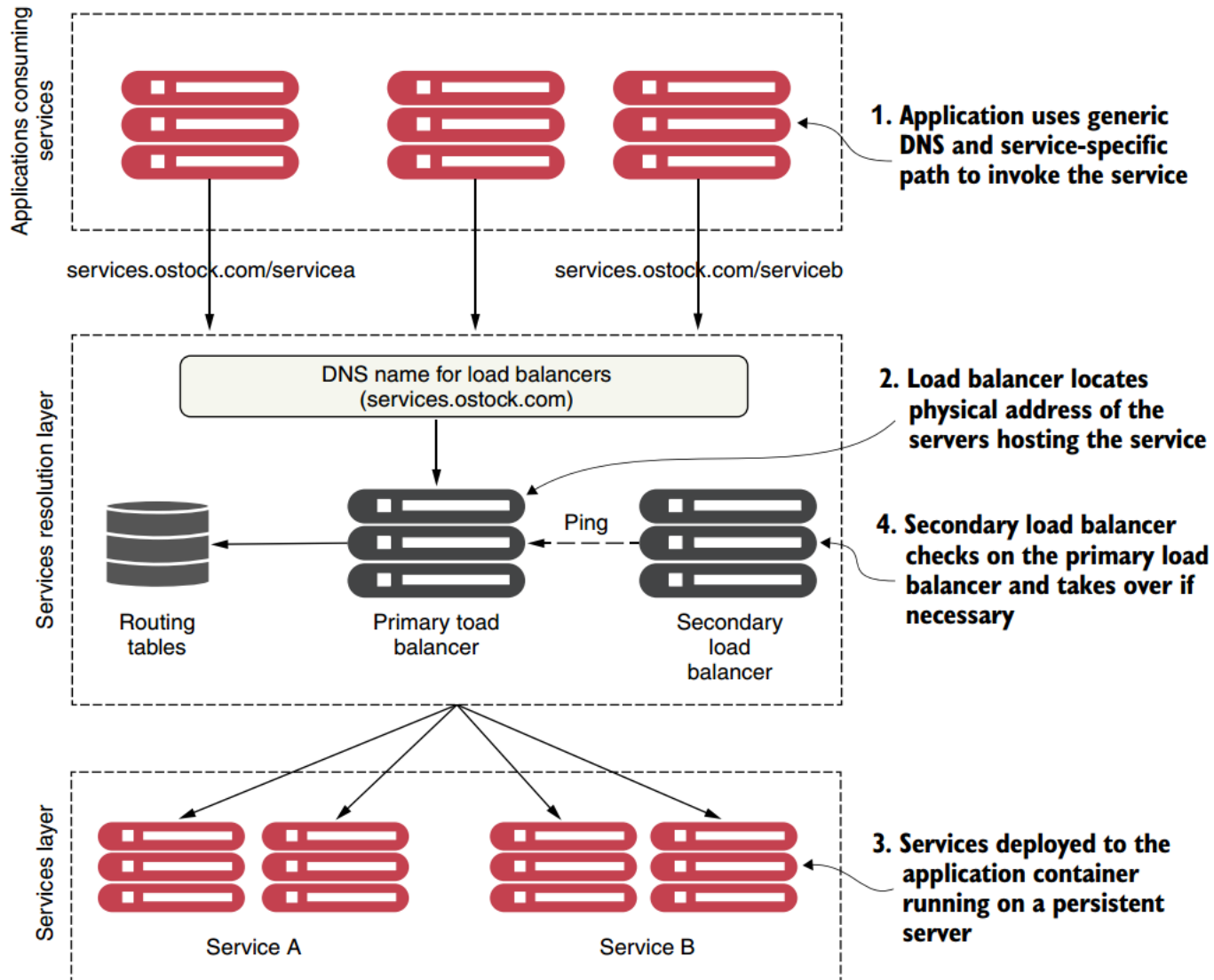


To scale more: Add more machines to existing **group of distributed system**

Client-side vs Server-side Service discovery

- Nós distinguimos entre duas abordagens para a descoberta de serviços:
 - Descoberta de serviço do lado do servidor (Server-side service discovery)
 - Descoberta de serviço do lado do cliente (Client-side service discovery)

Server-side load balancing com HAProxy/Nginx



Limitações do Server-side load balancing com HAProxy/Nginx

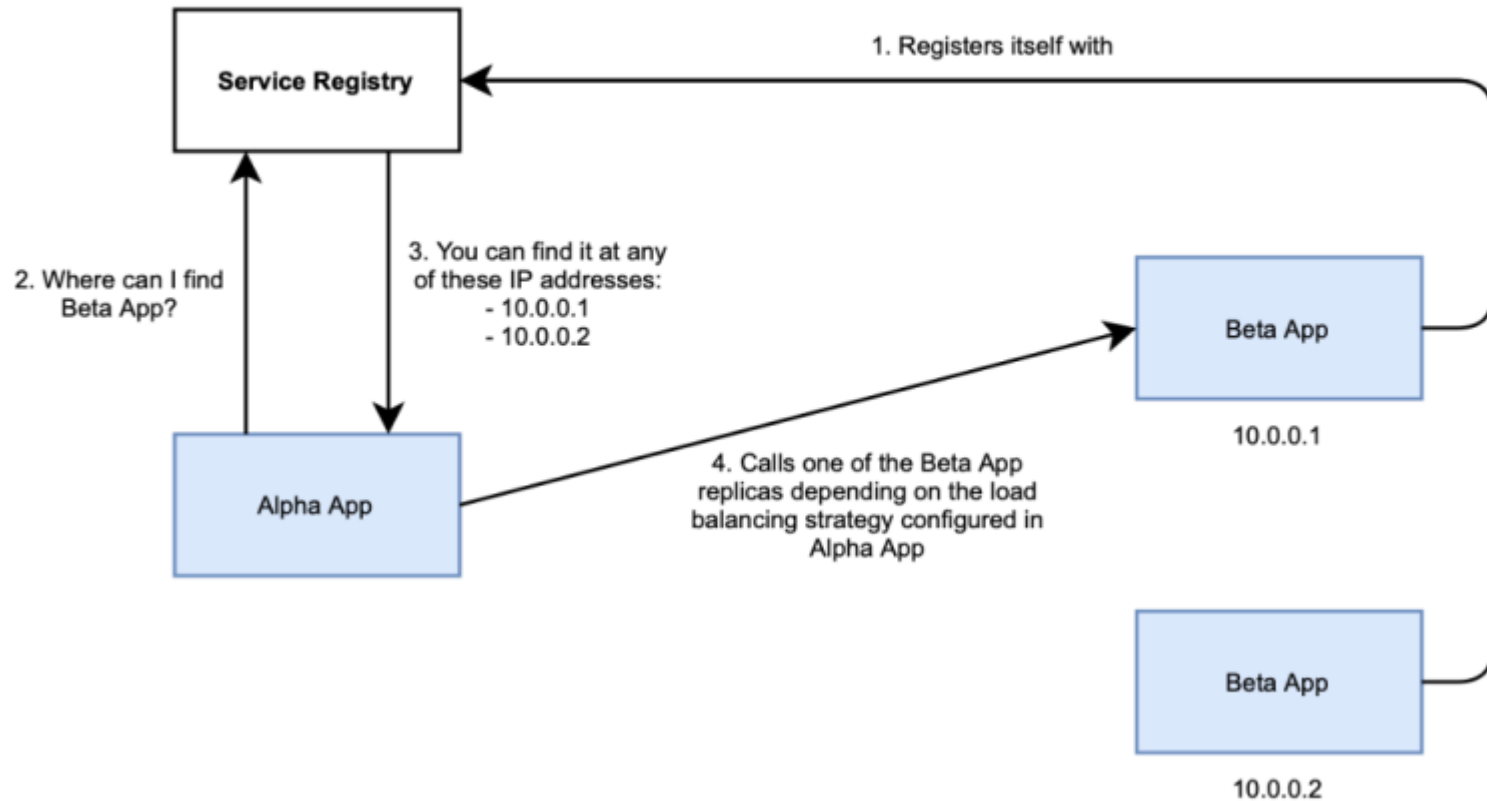
- Embora o balanceador de carga possa ser altamente disponível, **ele é um ponto único de falha para toda a sua infraestrutura**. Se o balanceador de carga falhar, todas as aplicações que dependem dele também falharão. Embora seja possível tornar um balanceador de carga altamente disponível, os balanceadores de carga tendem a ser pontos centralizados dentro da infraestrutura de aplicativos.
- Centralizar seus serviços em um único cluster de balanceadores de carga limita sua capacidade de escalar horizontalmente sua infraestrutura de balanceamento de carga em vários servidores. **Muitos balanceadores de carga comerciais são limitados por duas coisas: seu modelo de redundância e seus custos de licenciamento.**

Limitações do Server-side load balancing com HAProxy/Nginx

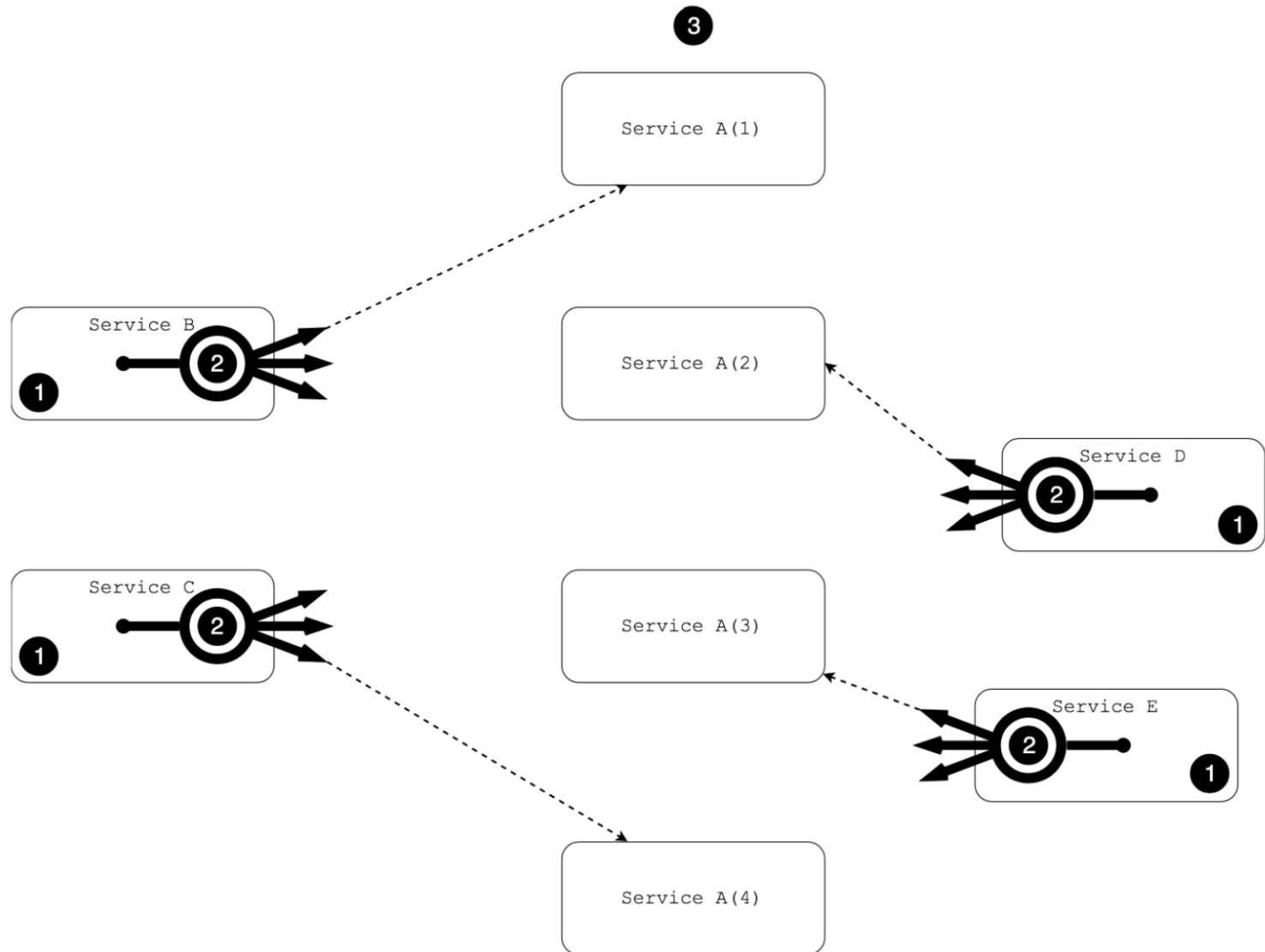
- **A maioria dos balanceadores de carga tradicionais é gerenciada de forma estática.** Eles não são projetados para registro e cancelamento rápidos de serviços. Os balanceadores de carga tradicionais usam um banco de dados centralizado para armazenar as rotas das regras, e a única forma de adicionar novas rotas geralmente é por meio da API proprietária do fornecedor.
- Como o balanceador de carga atua como um proxy para os serviços, as solicitações dos consumidores de serviços precisam ser mapeadas para os serviços físicos. **Essa camada de tradução frequentemente adiciona mais complexidade à infraestrutura de serviços, porque as regras de mapeamento para o serviço precisam ser definidas e implantadas manualmente.** Além disso, em um cenário de balanceador de carga tradicional, o registro de novas instâncias de serviço não é feito quando uma nova instância de serviço é iniciada.

Client-side load balancing

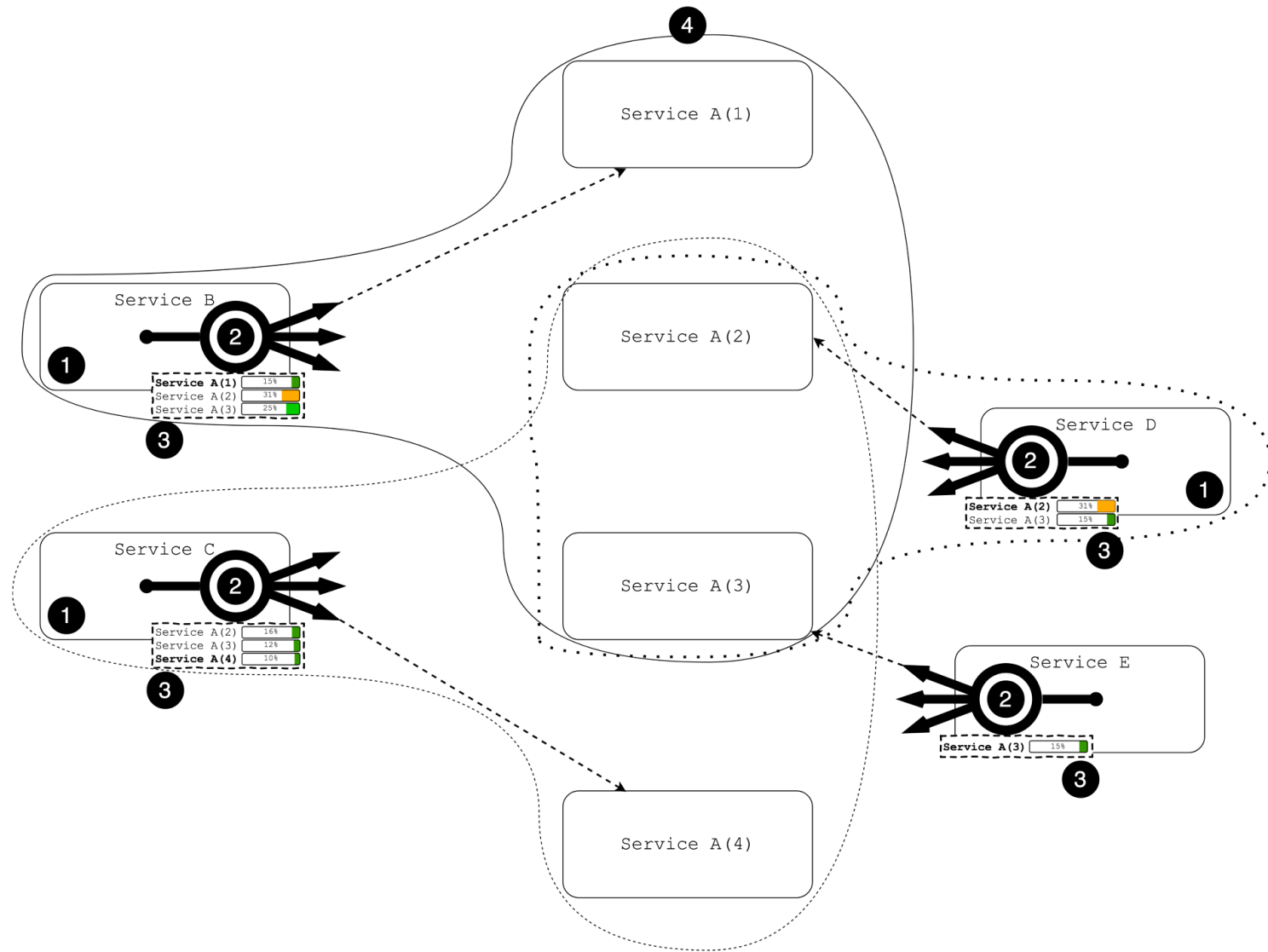
Client-side service discovery and load balancing - Model



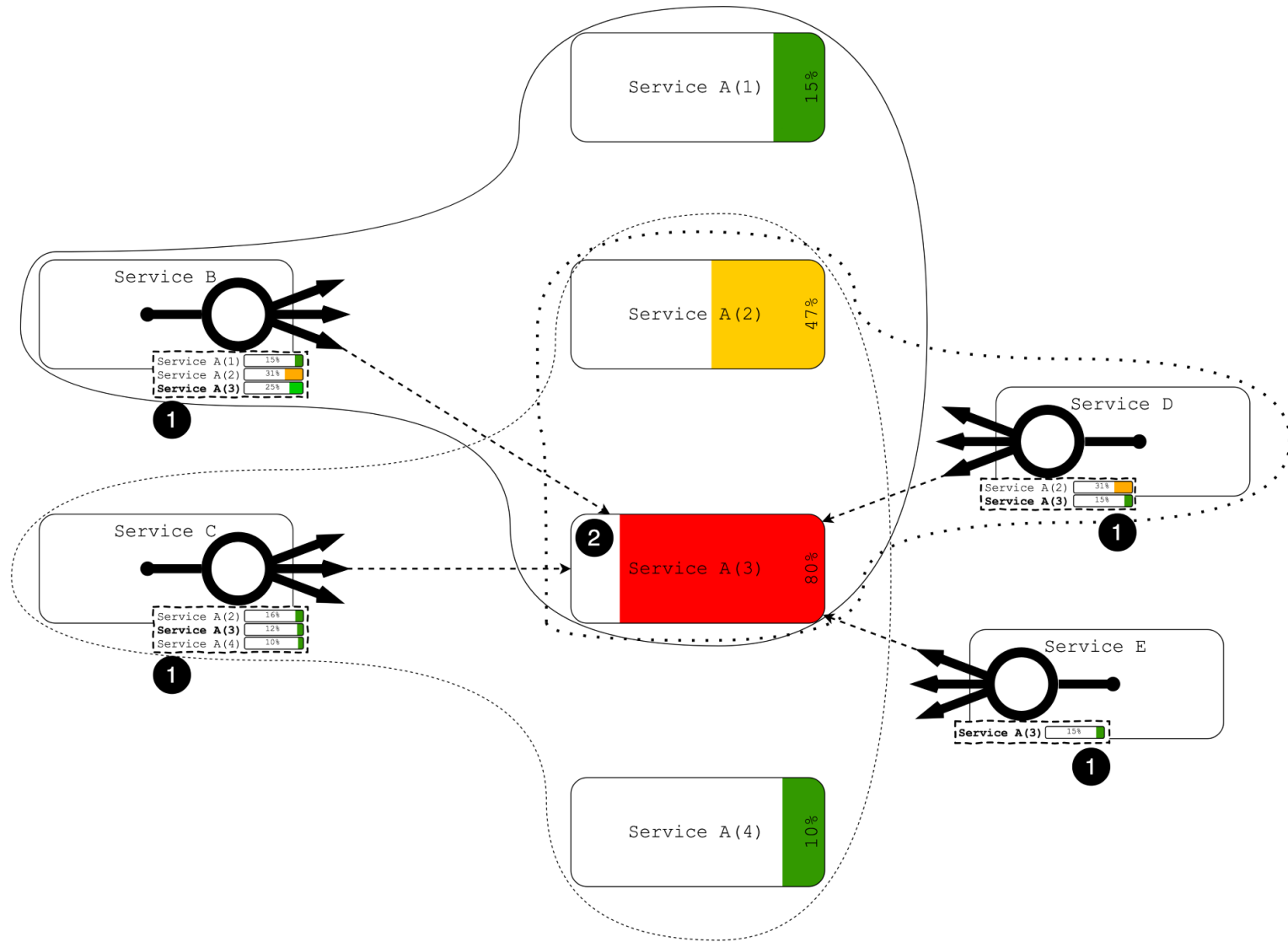
Client-side load balancing



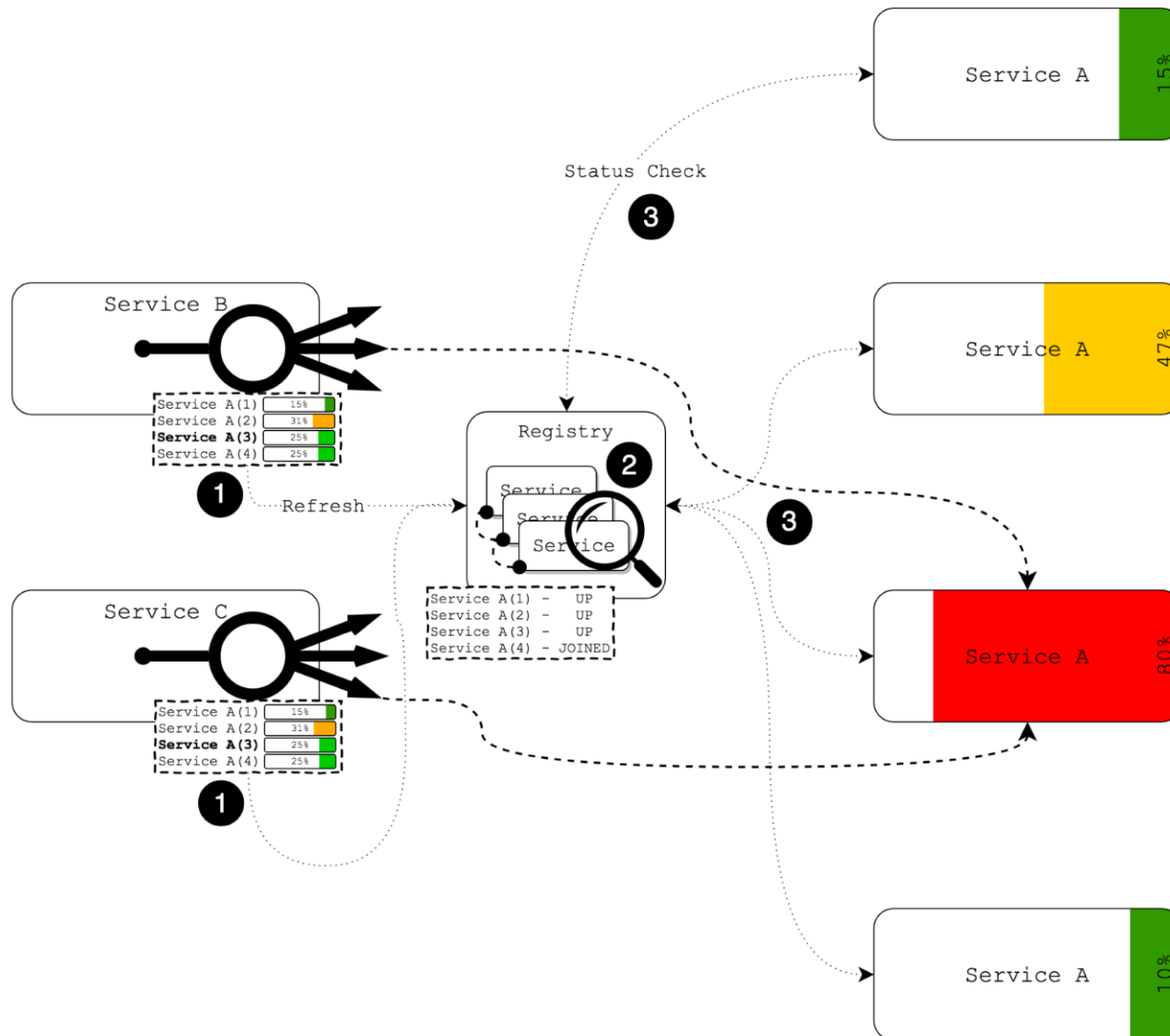
Client-side load balancing



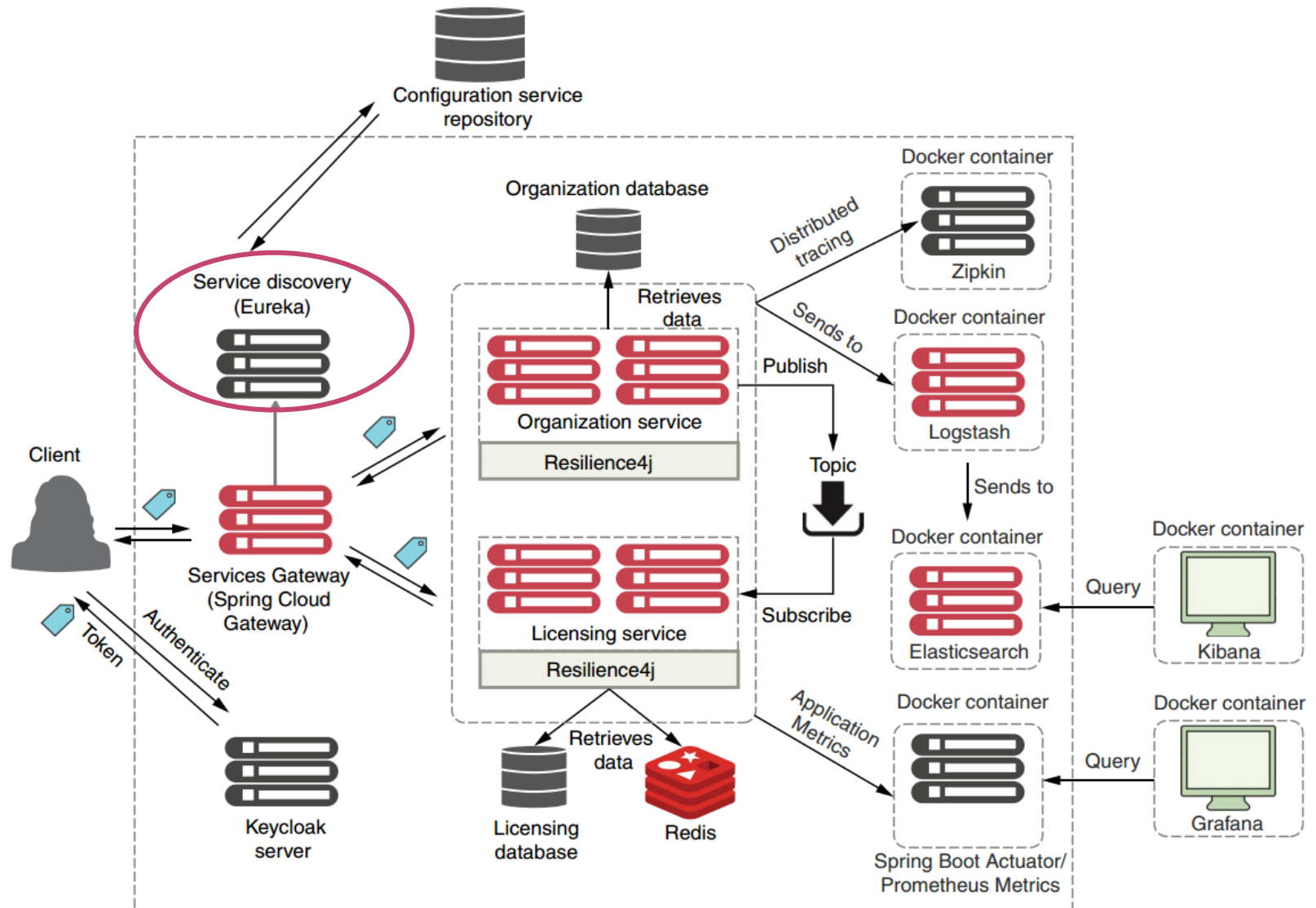
Client-side load balancing



Client-side load balancing



Microservice patterns: The Big Picture



Service Discovery in the cloud

- **Altamente disponível** — A descoberta de serviços precisa ser capaz de oferecer suporte a um ambiente de cluster “ativo” em que as buscas de serviços possam ser distribuídas entre vários nós de um cluster de descoberta de serviços. Se um nó ficar indisponível, os outros nós no cluster devem ser capazes de assumir suas funções.
- **Peer-to-peer** — Cada nó no cluster de descoberta de serviços compartilha o estado de uma instância de serviço.
- **Balanceado por carga** — A descoberta de serviços precisa balancear dinamicamente as solicitações entre todas as instâncias de serviço. Isso garante que as invocações de serviço sejam distribuídas entre todas as instâncias de serviço gerenciadas. De muitas maneiras, a descoberta de serviços substitui os balanceadores de carga mais estáticos e gerenciados manualmente que eram comuns em muitas implementações iniciais de aplicativos web.

Service Discovery in the cloud

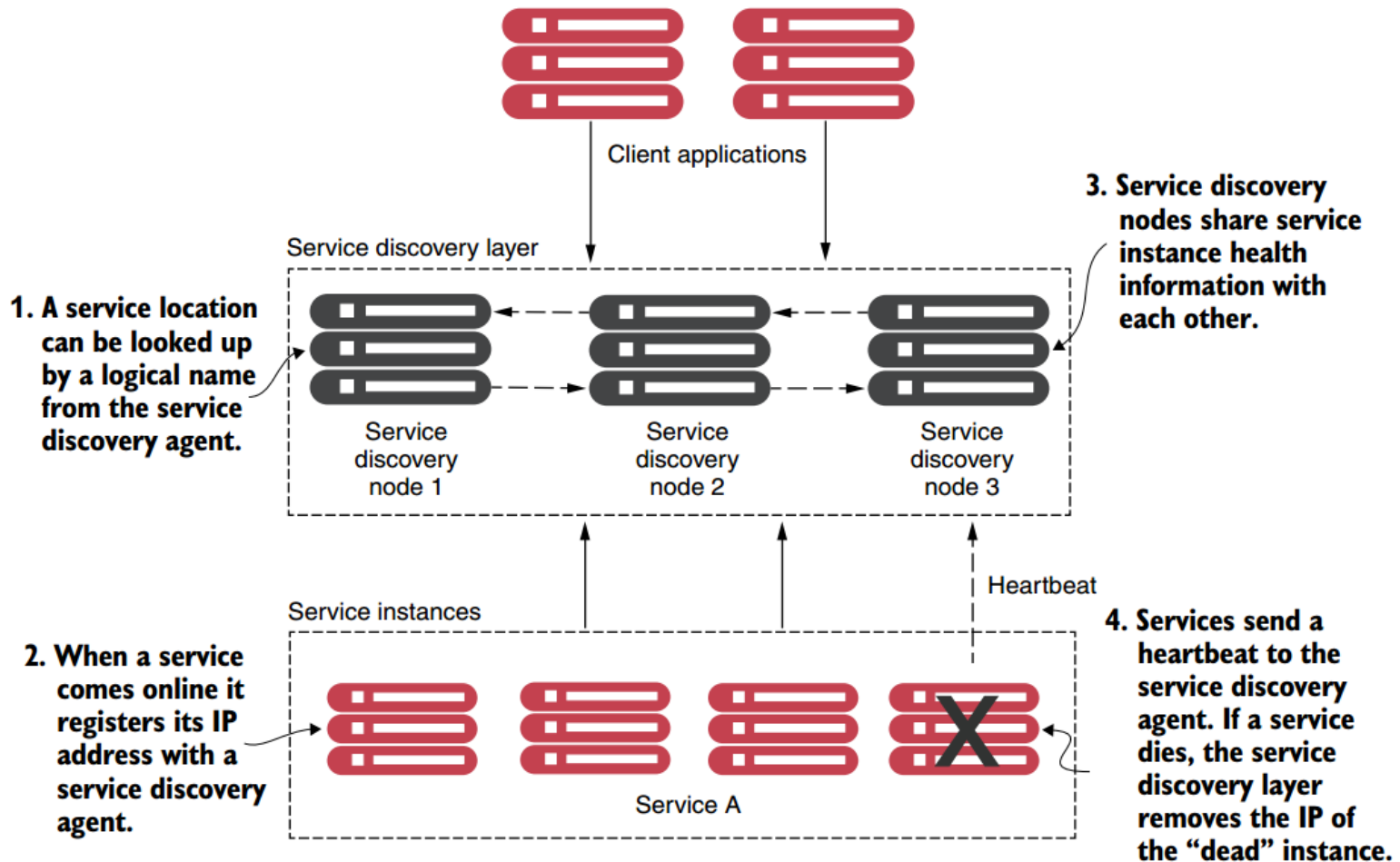
- **Resiliente** — O cliente da descoberta de serviços deve armazenar em cache as informações dos serviços **localmente**. O cache local permite uma degradação gradual da funcionalidade de descoberta de serviços, de forma que, se o serviço de descoberta de serviços ficar indisponível, os aplicativos ainda possam funcionar e localizar os serviços com base nas informações mantidas no cache local.
- **Tolerante a falhas** — A descoberta de serviços precisa detectar quando uma instância de serviço não está saudável e removê-la da lista de serviços disponíveis para atender às solicitações dos clientes. Deve ser capaz de identificar essas falhas nos serviços e agir automaticamente, sem necessidade de intervenção humana.

Architecture of Service Discovery

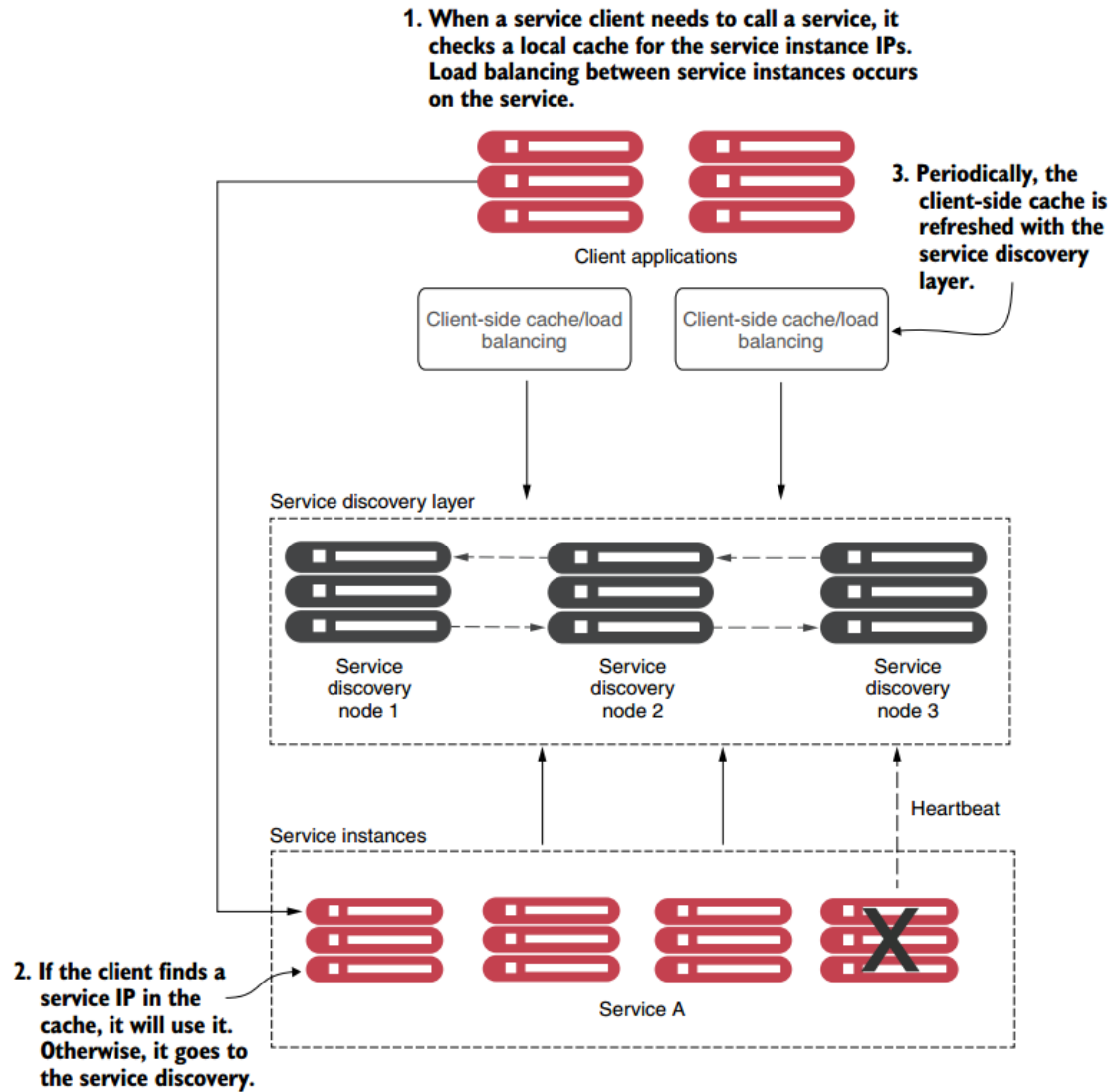
- O principal objetivo da descoberta de serviços é ter uma arquitetura onde os serviços indiquem sua localização física, em vez de configurar manualmente essa localização:
 - **Registro de serviço** — Como um serviço se registra com o agente de descoberta de serviços.
 - **Consulta de endereço do serviço pelo cliente** — Como um cliente de serviço busca informações sobre o serviço.
 - **Compartilhamento de informações** — Como os nós compartilham informações sobre os serviços.
 - **Monitoramento de saúde** — Como os serviços comunicam seu estado de saúde ao agente de descoberta de serviços.

Architecture of Service Discovery

Client applications never have direct knowledge of the IP address of a service. Instead, they get it from a service discovery agent.

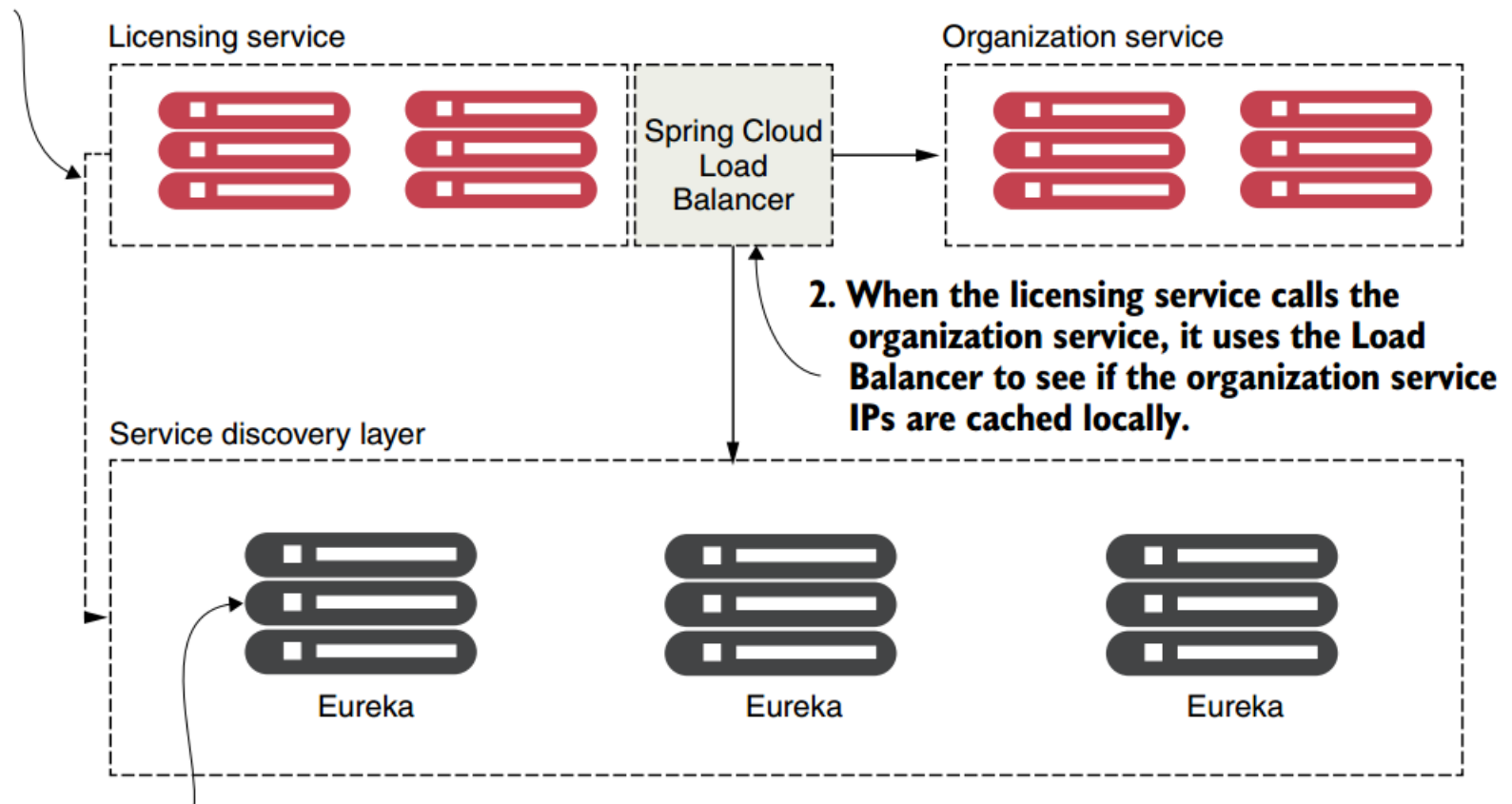


Service Discovery for Load balancing



Service Discovery for Load balancing

3. Periodically, Spring Cloud Load Balancer refreshes its cache of IP addresses.



2. When the licensing service calls the organization service, it uses the Load Balancer to see if the organization service IPs are cached locally.

1. As service instances start, they will register their IPs with Eureka.

Criando Naming Server

New Spring Starter Project 

Service URL:

Name:

☒ Use default location

Location:

Type: Packaging:

Java Version: Language:

Group:

Artifact:

Version:

Description:

Package:

Working sets

☐ Add project to working sets

Working sets:

Criando Naming Server

New Spring Starter Project Dependencies

Spring Boot Version: 2.5.0 (M3) ▾

Frequently Used:

☒ Config Client

☐ Config Server

☐ H2 Database

☒ Spring Boot Actuator

☒ Spring Boot DevTools

☐ Spring Data JPA

☐ Spring Web

Available:

eureka

Spring Cloud Discovery

☐ Eureka Discovery Client

☒ Eureka Server

VMware Tanzu Application Service

☐ Service Registry (TAS)

Selected:

X Spring Boot DevTools

X Spring Boot Actuator

X Config Client

X Eureka Server

Make Default

Clear Selection

?

< Back

Next >

Cancel

Finish

Criando Naming Server

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;

@SpringBootApplication
@EnableEurekaServer
public class NetflixEurekaNamingServerApplication {

    public static void main(String[] args) {
        SpringApplication.run(NetflixEurekaNamingServerApplication.class, args);
    }
}
```

Propriedades do Discovery Service

 application.properties 

```
1 spring.application.name=netflix-eureka-naming-server
2 server.port=8761
3 spring.config.import=optional:configserver:http://localhost:8888
4 eureka.client.register-with-eureka=false
5 eureka.client.fetch-registry=false
```


Propriedades do Discovery Service

- **registerWithEureka:** Essa propriedade controla se o cliente deve registrar-se no servidor Eureka como um serviço disponível.
 - Quando true: O cliente registra-se no servidor Eureka, informando que está disponível para atender a requisições. Esse é o comportamento padrão para serviços que querem ser descobertos por outros.
 - Quando false: O cliente não se registra no Eureka, sendo útil em casos como:
 - O cliente atua apenas como consumidor de serviços (não fornece serviços).
 - A aplicação é apenas um servidor Eureka (e não uma aplicação que fornece ou consome serviços).
 - A aplicação quer acessar os serviços registrados no Eureka sem anunciar sua presença.

Propriedades do Discovery Service

- **fetchRegistry:** Essa propriedade controla se o cliente deve buscar o registro de serviços no servidor Eureka.
 - Quando true: O cliente obtém informações de registro de outros serviços disponíveis no Eureka, permitindo-lhe localizar e consumir esses serviços.
 - Quando false: O cliente não busca o registro de serviços, sendo útil em casos como:
 - A aplicação é apenas um serviço que se registra no Eureka e não consome outros serviços.
 - A aplicação é o servidor Eureka principal ou um nó em cluster, onde não precisa buscar informações de outros serviços no registro (porque ela própria é o registro).

← → ↻ ⓘ localhost:8761

☆

🌐

🔌

📶

🔊

🔋

🔍

⌵

🔒

Atualizar

spring Eureka

HOME

LAST 1000 SINCE STARTUP

System Status

Environment	N/A
Data center	N/A

Current time	2021-04-08T09:24:41 -0300
Uptime	00:00
Lease expiration enabled	false
Renews threshold	1
Renews (last min)	0

DS Replicas

localhost

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
No instances available			

General Info

Name	Value
total-avail-memory	100mb
num-of-cpus	8
current-memory-usage	40mb (40%)
server-uptime	00:00
registered-replicas	http://localhost:8761/eureka/
unavailable-replicas	http://localhost:8761/eureka/
available-replicas	

Instance Info

Name	Value
ipAddr	192.168.0.51
status	UP

Acessando o Naming Server

DS Replicas

Instances currently registered with Eureka

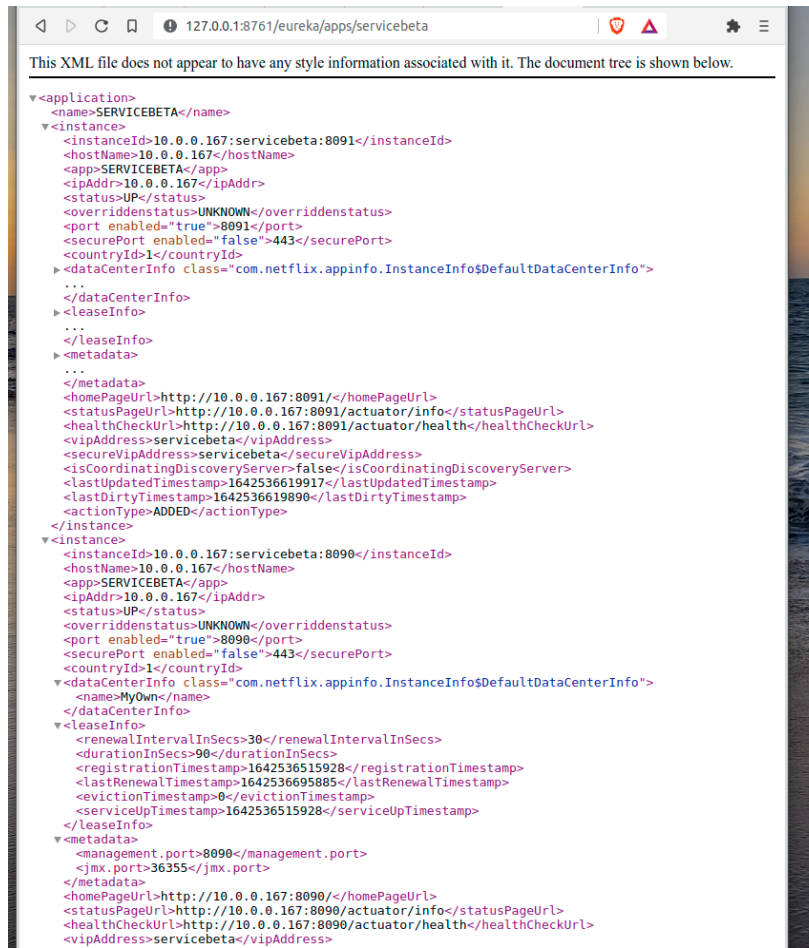
Application	AMIs	Availability Zones	Status
SERVICEBETA	n/a (2)	(2)	UP (2) - 10.0.0.167:servicebeta:8091 , 10.0.0.167:servicebeta:8090

General Info

Name	Value
total-avail-memory	82mb
num-of-cpus	8
current-memory-usage	42mb (51%)
server-uptime	00:04
registered-replicas	
unavailable-replicas	
available-replicas	

Acessando o Discovery Server

- Eureka's REST API:
 - `http://<eureka service>:8070/eureka/apps/<APPID>`



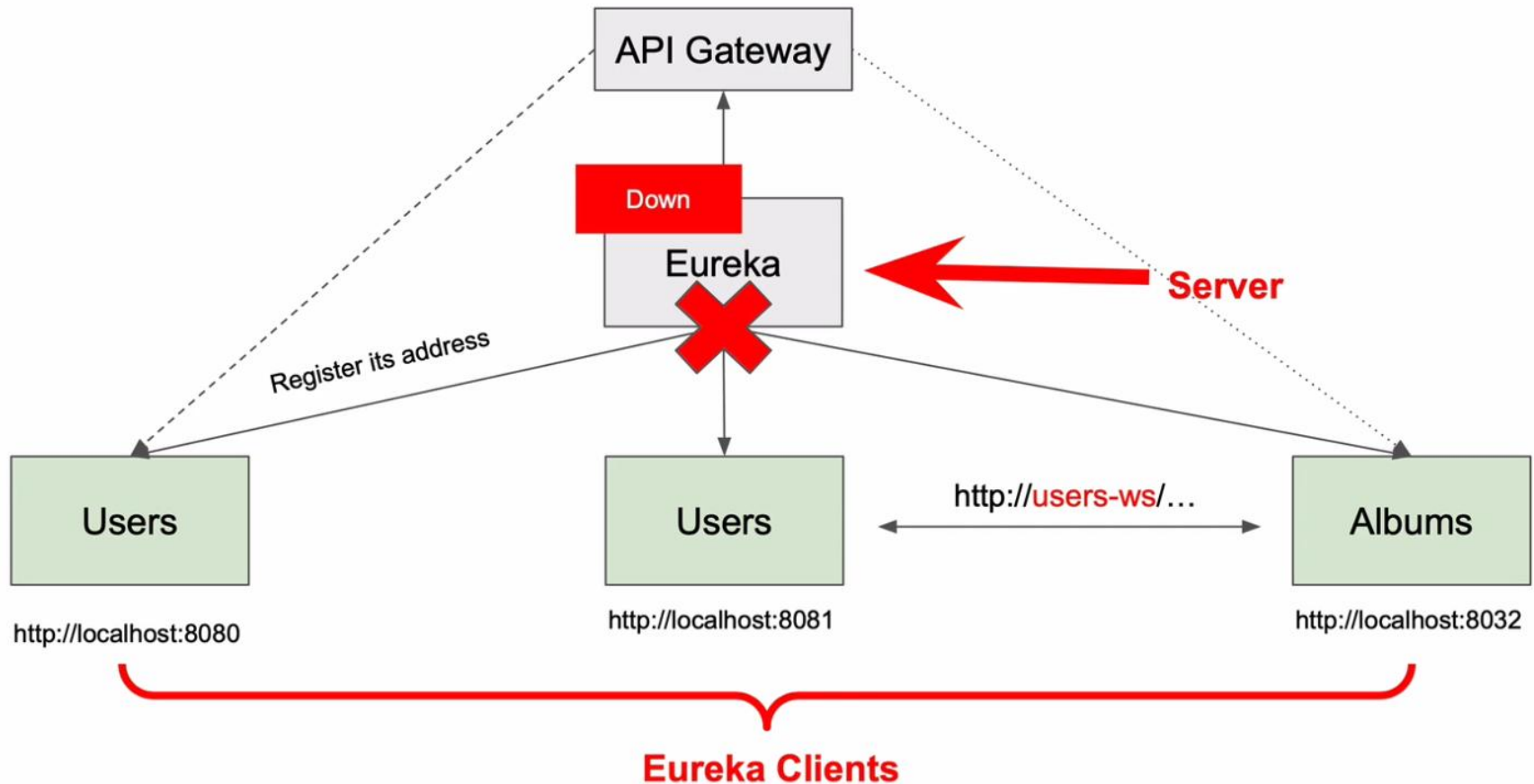
The screenshot shows a web browser window with the address bar displaying `127.0.0.1:8761/eureka/apps/servicebeta`. The page content indicates that the XML file does not have any style information associated with it, and the document tree is shown below. The XML content is as follows:

```
<?xml version="1.0"?>
<application>
  <name>SERVICEBETA</name>
  <instance>
    <instanceId>10.0.0.167:servicebeta:8091</instanceId>
    <hostName>10.0.0.167</hostName>
    <app>SERVICEBETA</app>
    <ipAddr>10.0.0.167</ipAddr>
    <status>UP</status>
    <overrides><override status="UNKNOWN" /></overrides>
    <port enabled="true">8091</port>
    <securePort enabled="false">443</securePort>
    <countryId>1</countryId>
    <dataCenterInfo class="com.netflix.appinfo.InstanceInfo$DefaultDataCenterInfo">
      <name>DefaultDataCenterInfo</name>
    </dataCenterInfo>
  </instance>
  <instance>
    <instanceId>10.0.0.167:servicebeta:8090</instanceId>
    <hostName>10.0.0.167</hostName>
    <app>SERVICEBETA</app>
    <ipAddr>10.0.0.167</ipAddr>
    <status>UP</status>
    <overrides><override status="UNKNOWN" /></overrides>
    <port enabled="true">8090</port>
    <securePort enabled="false">443</securePort>
    <countryId>1</countryId>
    <dataCenterInfo class="com.netflix.appinfo.InstanceInfo$DefaultDataCenterInfo">
      <name>DefaultDataCenterInfo</name>
    </dataCenterInfo>
  </instance>
  <leaseInfo>
    <renewalIntervalInSecs>30</renewalIntervalInSecs>
    <durationInSecs>90</durationInSecs>
    <registrationTimestamp>1642536515928</registrationTimestamp>
    <lastRenewalTimestamp>1642536515928</lastRenewalTimestamp>
    <evictionTimestamp>0</evictionTimestamp>
    <serviceUpTimestamp>1642536515928</serviceUpTimestamp>
  </leaseInfo>
  <metadata>
    <management.port>8090</management.port>
    <jmx.port>36355</jmx.port>
  </metadata>
  <homePageUrl>http://10.0.0.167:8090/</homePageUrl>
  <statusPageUrl>http://10.0.0.167:8090/actuator/info</statusPageUrl>
  <healthCheckUrl>http://10.0.0.167:8090/actuator/health</healthCheckUrl>
  <vipAddress>servicebeta</vipAddress>
  <secureVipAddress>servicebeta</secureVipAddress>
  <isCoordinatingDiscoveryServer>false</isCoordinatingDiscoveryServer>
  <lastUpdatedTimestamp>1642536619917</lastUpdatedTimestamp>
  <lastDirtyTimestamp>1642536619917</lastDirtyTimestamp>
  <actionType>ADDED</actionType>
</application>
```

Obtendo instâncias usando o Spring Discovery Client

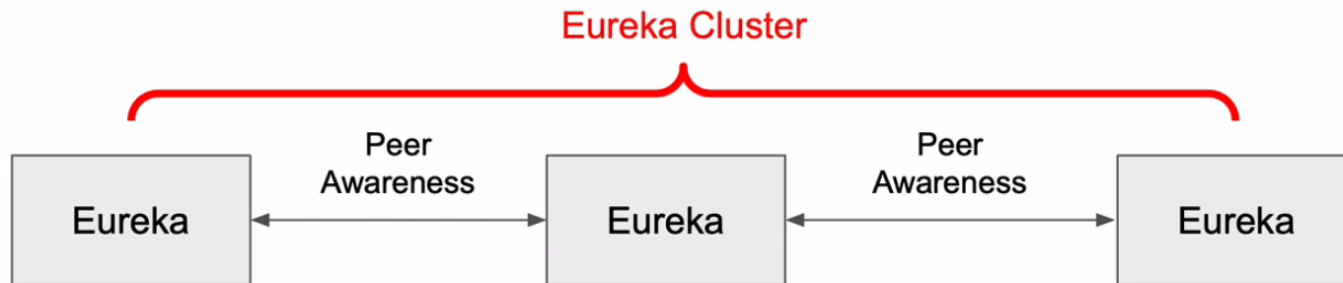
```
6 import org.springframework.beans.factory.annotation.Autowired;
7 import org.springframework.cloud.client.ServiceInstance;
8 import org.springframework.cloud.client.discovery.DiscoveryClient;
9 import org.springframework.stereotype.Component;
10 import org.springframework.web.client.RestTemplate;
11
12 @Component
13 public class BetaRestTemplateClient {
14     @Autowired
15     private DiscoveryClient discoveryClient;
16
17     public void listBetaServices() {
18         List<ServiceInstance> instances = discoveryClient.getInstances("SERVICEBETA");
19         if (instances.size() == 0)
20             return;
21         for (Iterator iterator = instances.iterator(); iterator.hasNext();) {
22             ServiceInstance serviceInstance = (ServiceInstance) iterator.next();
23             System.out.println(serviceInstance.getHost() + ":" + serviceInstance.getPort());
24         }
25     }
26 }
```

Limitações do Modo Standalone

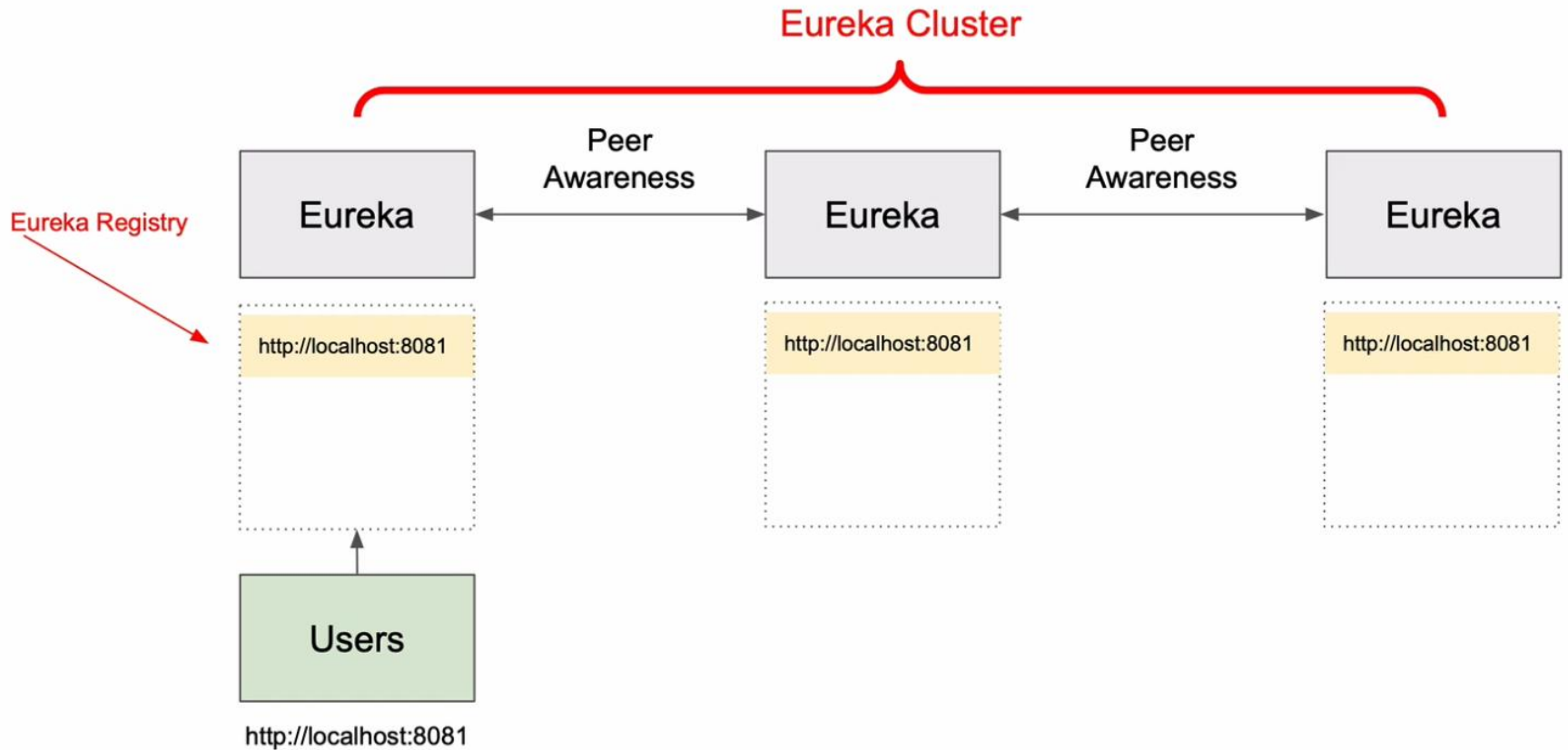


Em ambiente de Produção devemos evitar um ponto de falha!

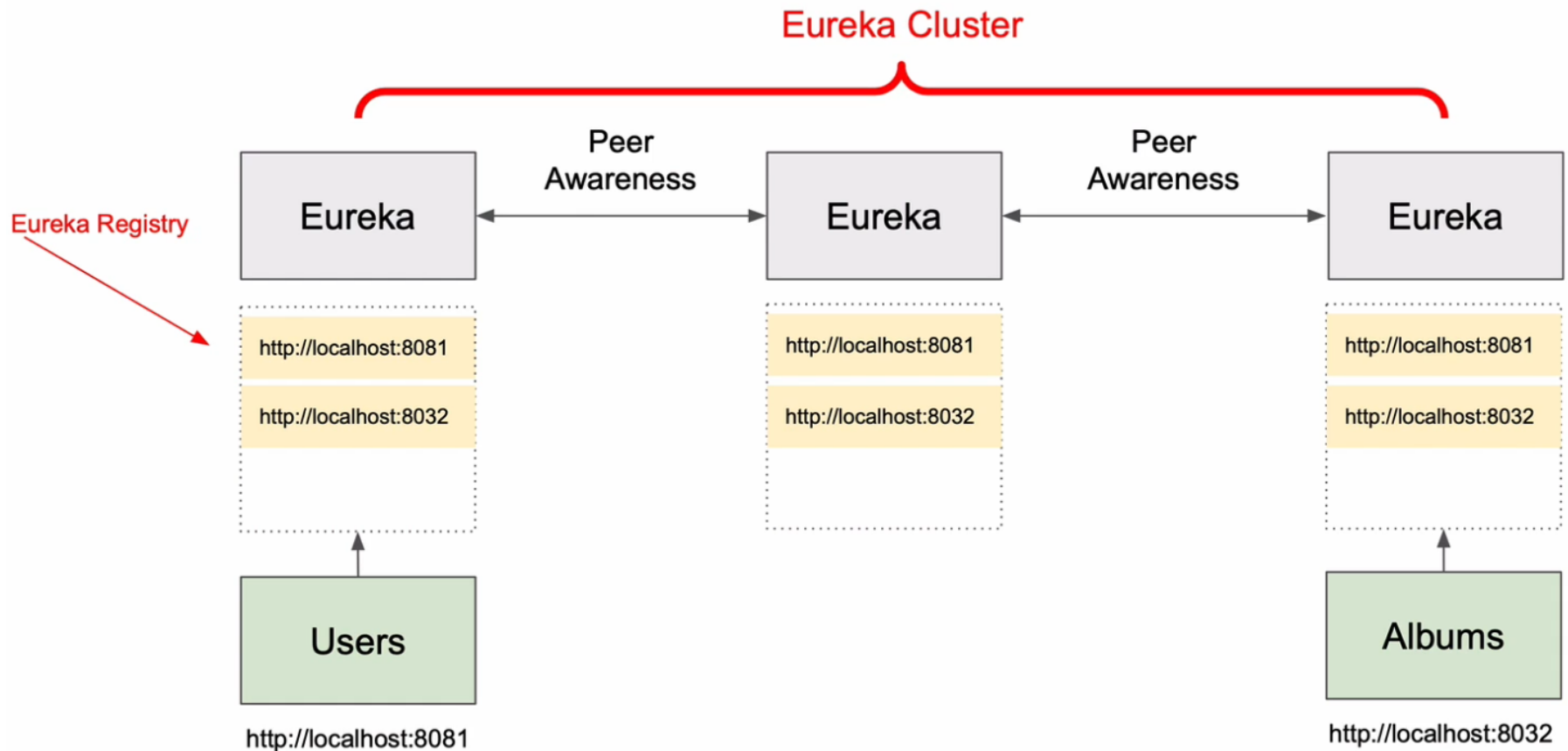
- Para resolver o problema de 1 ponto de falha para o caso do Eureka Server falhar, devemos usar um cluster de Eureka servers.



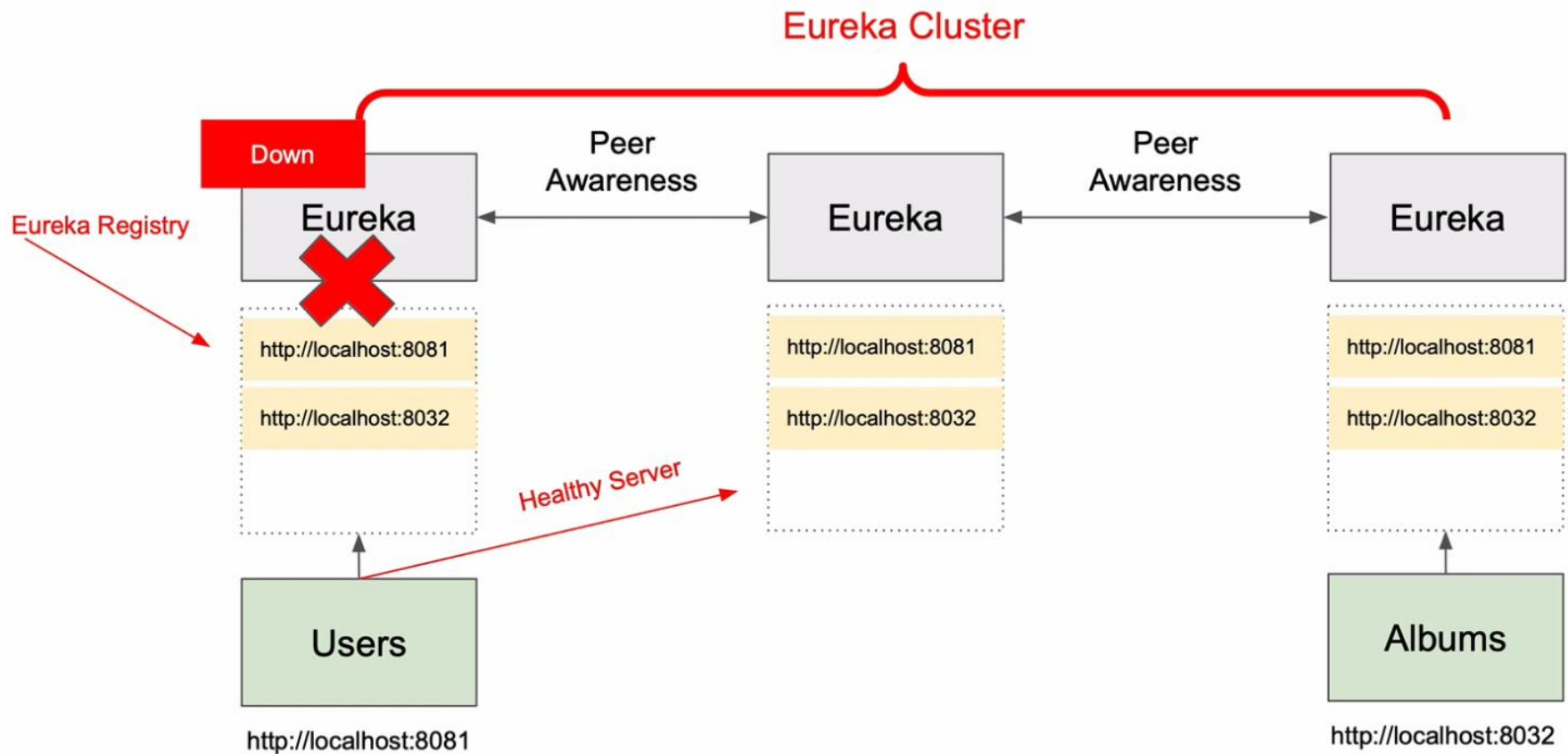
Operação do Eureka em Cluster



Operação do Eureka em Cluster



Operação do Eureka em Cluster



Definindo o Cluster de Eureka

- Para definir o cluster de Eureka, é preciso executar várias instância do mesmo serviço. Para tanto, podemos usar essa abordagem de criar vários profiles, um para cada instância.

- ✦  src/main/resources
 -  application.properties
 -  application-server1.properties
 -  application-server2.properties
 -  application-server3.properties

Definindo o Cluster de Eureka

- Primeiro podemos usar o application.Properties

application.properties ×

```
1 spring.application.name=ClusterEurekaServer
```

- Na sequencia, devemos definir um profile para cada instância de Eureka

application-server1.properties ×

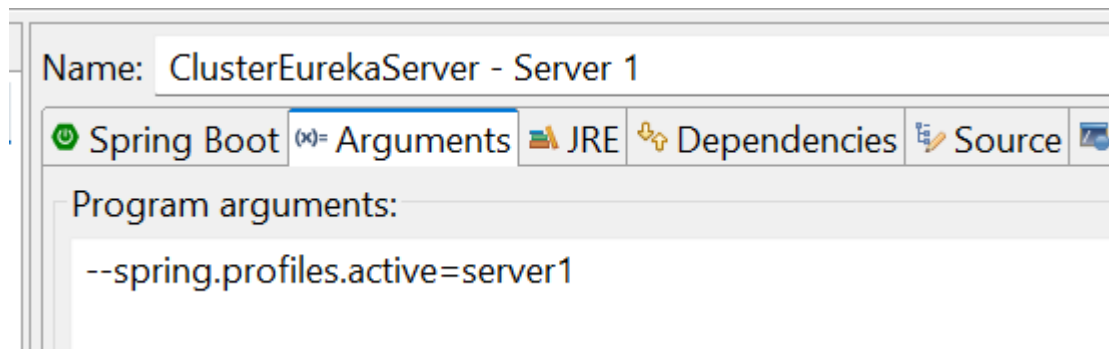
```
1 server.port = 8761  
2 eureka.instance.hostname=server1  
3 eureka.client.serviceUrl.defaultZone=http://server2:8762/eureka,http://server3:8763/eureka
```

Definindo o Cluster de Eureka

- Para um cluster de 3 instâncias, teríamos ainda:

```
application-server2.properties ×  
1 server.port = 8762  
2 eureka.instance.hostname=server2  
3 eureka.client.serviceUrl.defaultZone=http://server1:8761/eureka,http://server3:8763/eureka  
  
application-server3.properties ×  
1 server.port = 8763  
2 eureka.instance.hostname=server3  
3 eureka.client.serviceUrl.defaultZone=http://server1:8761/eureka,http://server2:8762/eureka
```

- Para executar no Eclipse, por exemplo:



Usando o Cluster de Eureka no Microserviço Cliente

```
1 spring.application.name= servicebeta
2 server.port=8090
3 spring.cloud.loadbalancer.ribbon.enabled = false
4 eureka.instance.preferIpAddress = true
5 eureka.client.registerWithEureka = true
6 eureka.client.fetchRegistry = true
7 eureka.client.serviceUrl.defaultZone = http://server1:8761/eureka,http://server2:8762/eureka,http://server3:8763/eureka
```

Spring Cloud Cloud LoadBalancer

- O balanceamento de carga é o processo de distribuir o tráfego entre diferentes instâncias de uma mesma aplicação.
- Para criar um sistema tolerante a falhas, é comum executar várias instâncias de cada aplicação.
- Assim, sempre que um serviço precisar se comunicar com outro, ele deve escolher uma instância específica para enviar sua solicitação.
- O **Spring Cloud Load Balancer** substituiu o balanceador de carga Ribbon.

Spring Cloud Cloud LoadBalancer

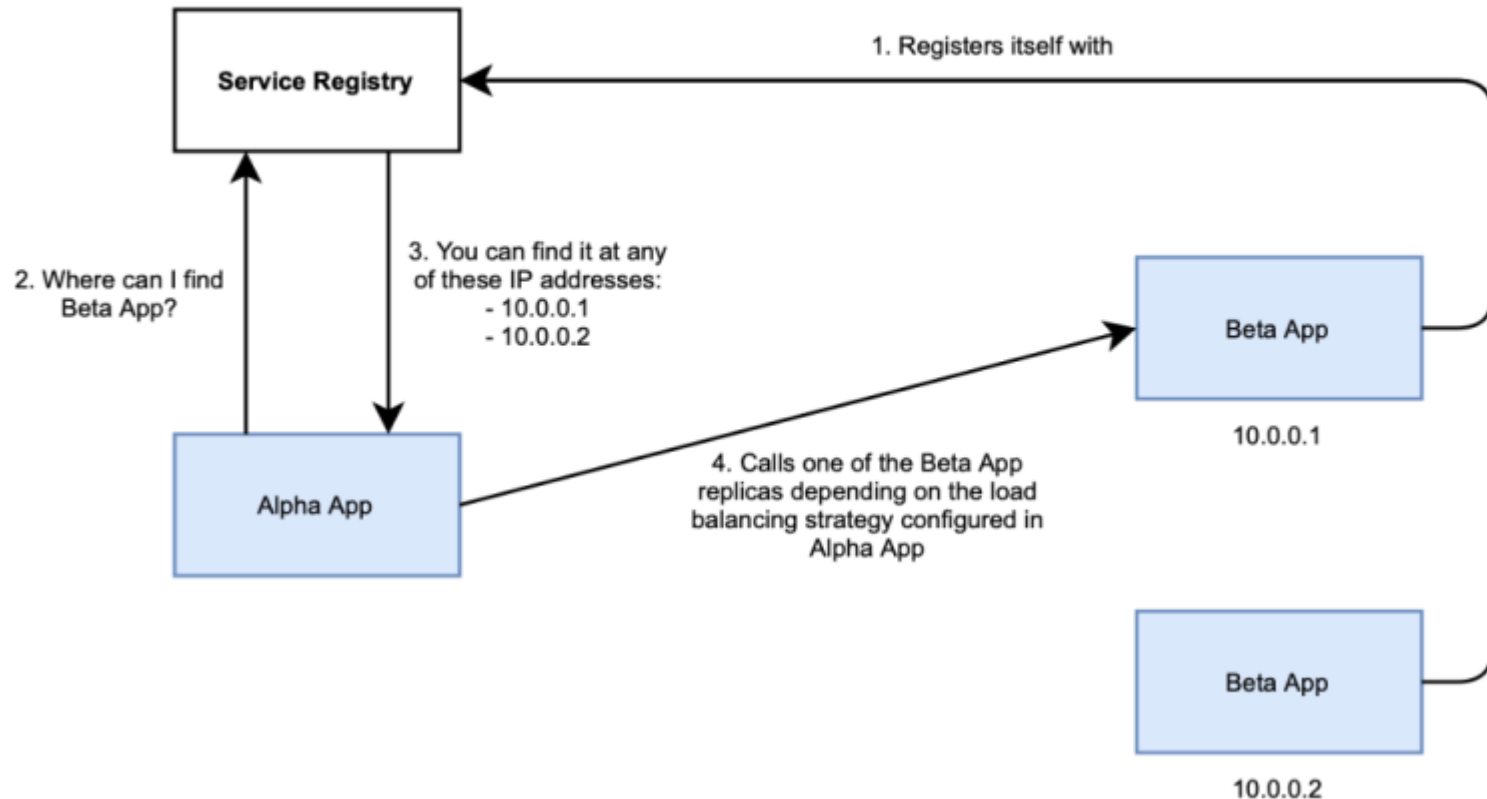
```
<dependency>  
  <groupId>org.springframework.cloud</groupId>  
  <artifactId>spring-cloud-starter-loadbalancer</artifactId>  
</dependency>
```

Como o Ribbon ainda é o balanceador de carga padrão no lado do cliente, desativamos essa funcionalidade utilizando:

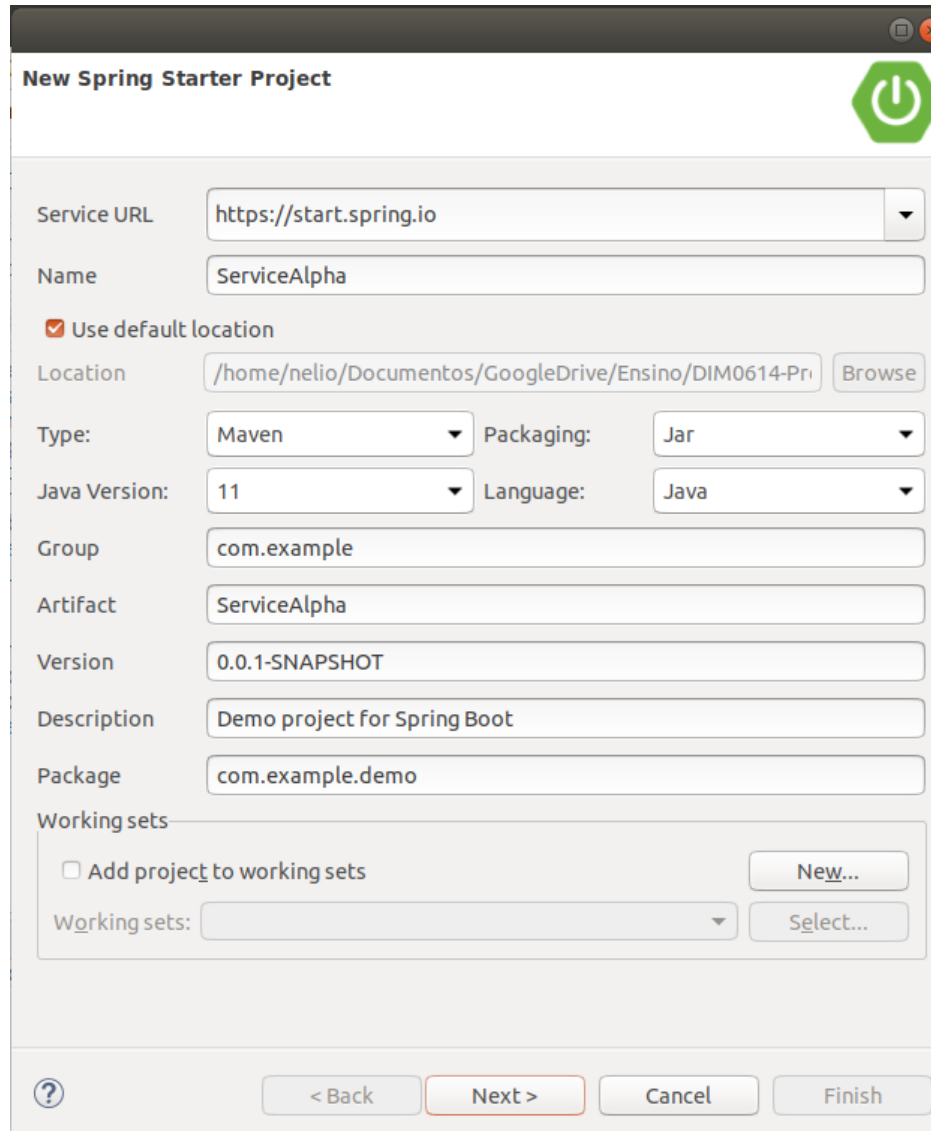
spring.cloud.loadbalancer.ribbon.enabled=false

Example of Client-side load balancing

Client-side service discovery and load balancing - Model



Usando LoadBalancer in Spring



New Spring Starter Project

Service URL:

Name:

☒ Use default location

Location:

Type: Packaging:

Java Version: Language:

Group:

Artifact:

Version:

Description:

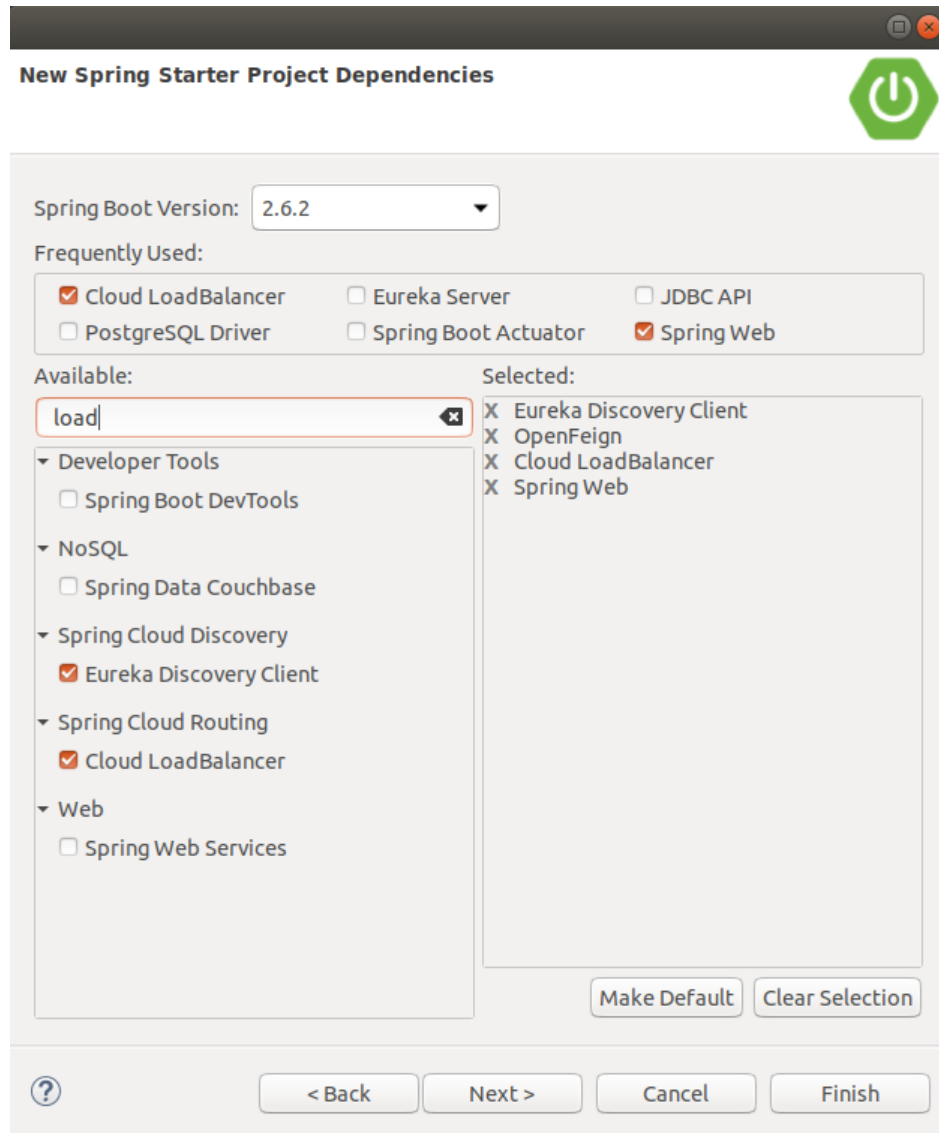
Package:

Working sets

☐ Add project to working sets

Working sets:

Using LoadBalancer in Spring



New Spring Starter Project Dependencies

Spring Boot Version: 2.6.2

Frequently Used:

- ☒ Cloud LoadBalancer
- ☐ PostgreSQL Driver
- ☐ Eureka Server
- ☐ Spring Boot Actuator
- ☐ JDBC API
- ☒ Spring Web

Available:

- Developer Tools
 - ☐ Spring Boot DevTools
- NoSQL
 - ☐ Spring Data Couchbase
- Spring Cloud Discovery
 - ☒ Eureka Discovery Client
- Spring Cloud Routing
 - ☒ Cloud LoadBalancer
- Web
 - ☐ Spring Web Services

Selected:

- X Eureka Discovery Client
- X OpenFeign
- X Cloud LoadBalancer
- X Spring Web

load

Make Default Clear Selection

< Back Next > Cancel Finish

Criando Service Alpha

```
1 spring.application.name= servicealpha
2 server.port= 8081
3 spring.cloud.loadbalancer.ribbon.enabled = false
4 eureka.instance.preferIpAddress = true
5 eureka.client.registerWithEureka = true
6 eureka.client.fetchRegistry = true
7 eureka.client.serviceUrl.defaultZone = http://localhost:8761/eureka/
```

eureka.instance.preferIpAddress = true	←	Registers the IP address of the service rather than the server name
eureka.client.registerWithEureka = true	←	Registers the service with Eureka
eureka.client.fetchRegistry = true	←	Pulls down a local copy of the registry
eureka.client.serviceUrl.defaultZone = http://localhost:8070/eureka/	←	Sets the location of the Eureka service

Criando Service Alpha

```
@SpringBootApplication
@EnableDiscoveryClient
public class ServiceAlphaApplication {

    public static void main(String[] args) {
        SpringApplication.run(ServiceAlphaApplication.class, args);
    }

    @LoadBalanced
    @Bean
    public RestTemplate getRestTemplate(){
        return new RestTemplate();
    }
}
```

Criando Service Alpha

```
@RestController
@RequestMapping("/alpha")
public class AlphaController {

    @Autowired
    RestTemplate restTemplate;

    @GetMapping
    public ResponseEntity getData() {
        return restTemplate.getForEntity("http://SERVICEBETA/beta", String.class);
    }
}
```

Criando Service Beta

New Spring Starter Project 

Service URL:

Name:

☒ Use default location

Location:

Type: Packaging:

Java Version: Language:

Group:

Artifact:

Version:

Description:

Package:

Working sets

☐ Add project to working sets

Working sets:

Criando Service Beta

New Spring Starter Project Dependencies

Spring Boot Version: 2.6.2

Frequently Used:

☒ Cloud LoadBalancer

☐ JDBC API

☐ Spring Boot Actuator

☒ Eureka Discovery Client

☐ OpenFeign

☒ Spring Web

☐ Eureka Server

☐ PostgreSQL Driver

Available:

Type to search dependencies

- Developer Tools
- Google Cloud Platform
- I/O
- Messaging
- Microsoft Azure
- NoSQL
- Observability
- Ops
- SQL
- Security
- Spring Cloud
- Spring Cloud Circuit Breaker
- Spring Cloud Config
- Spring Cloud Discovery

Selected:

X Eureka Discovery Client

X Cloud LoadBalancer

X Spring Web

Make Default

Clear Selection



< Back

Next >

Cancel

Finish

Criando Service Beta

```
1 spring.application.name= servicebeta
2 server.port= 8090
3 spring.cloud.loadbalancer.ribbon.enabled = false
4 eureka.instance.preferIpAddress = true
5 eureka.client.registerWithEureka = true
6 eureka.client.fetchRegistry = true
7 eureka.client.serviceUrl.defaultZone = http://localhost:8761/eureka/
```

Criando Service Beta

```
@SpringBootApplication
@EnableDiscoveryClient
public class ServiceBetaApplication {

    public static void main(String[] args) {
        SpringApplication.run(ServiceBetaApplication.class, args);
    }
}
```

Criando Service Beta

```
@RestController
@RequestMapping("/beta")
public class BetaController {

    @Value("${server.port}")
    private int serverPort;

    @GetMapping
    public ResponseEntity listStudents() {
        return ResponseEntity.ok("OK from "+serverPort);
    }
}
```

Executando o Ambiente

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
SERVICEALPHA	n/a (1)	(1)	UP (1) - 10.0.0.167:servicealpha:8081
SERVICEBETA	n/a (2)	(2)	UP (2) - 10.0.0.167:servicebeta:8091 , 10.0.0.167:servicebeta:8090



Invocando serviços com Netflix Feign client

- Uma alternativa à classe **RestTemplate** habilitada pelo **Spring Load Balancer** é a biblioteca **Feign** da Netflix.
- A biblioteca Feign adota uma abordagem diferente para consumir um serviço REST.
- Nessa abordagem, o desenvolvedor primeiro define uma interface Java e, em seguida, adiciona anotações do **Spring Cloud** para mapear qual serviço baseado no **Eureka** o **Spring Cloud Load Balancer** irá invocar.
- O framework **Spring Cloud** gerará dinamicamente uma classe proxy para invocar o serviço REST direcionado.

Invocando serviços com Netflix Feign client

```
@SpringBootApplication
@EnableDiscoveryClient
@EnableFeignClients
public class ServiceAlphaApplication {

    public static void main(String[] args) {
        SpringApplication.run(ServiceAlphaApplication.class, args);
    }

    @LoadBalanced
    @Bean
    public RestTemplate getRestTemplate(){
        return new RestTemplate();
    }
}
```

Invocando serviços com Netflix Feign client

Defina uma interface Java e, em seguida, adiciona anotações do Spring Cloud para mapear qual serviço baseado no Eureka será invocado pelo Spring Cloud Load Balancer.

O framework Spring Cloud irá gerar dinamicamente uma classe proxy para invocar o serviço REST direcionado.

```
3 import org.springframework.cloud.openfeign.FeignClient;
4 import org.springframework.web.bind.annotation.RequestMapping;
5 import org.springframework.web.bind.annotation.RequestMethod;
6
7 @FeignClient("servicebeta")
8 public interface BetaServiceInterface {
9     @RequestMapping(method= RequestMethod.GET, value="/beta", consumes="application/json") String getData();
10 }
11
12 |
```


Invocando serviços com Netflix Feign client

```
12 @RestController
13 @RequestMapping("/alpha")
14 public class AlphaController {
15
16
17     @Autowired
18     BetaServiceInterface betaservice;
19
20     @GetMapping
21     public ResponseEntity getData() {
22         return ResponseEntity.ok(betaservice.getData());
23     }
24 }
25
26
```

HTTP Interface

- **Introduzido no Spring Framework 6**
- A HTTP Interface inclui métodos anotados para invocação de endpoints HTTP.
- Podemos simplesmente expressar os detalhes da API remota usando uma interface Java anotada e permitir que o Spring gere um proxy que implementa essa interface e realiza as trocas.
- `@HttpExchange` é a anotação raiz que podemos aplicar a uma interface HTTP e seus métodos de troca. No caso de aplicarmos essa anotação no nível da interface, ela será aplicada a todos os métodos. Isso pode ser útil para especificar atributos comuns a todos os métodos da interface, como tipo de conteúdo ou prefixo de URL.

HTTP Interface

- Existem outras anotações adicionais disponíveis para todos os métodos HTTP:
 - @GetExchange para requisições HTTP GET
 - @PostExchange para requisições HTTP POST
 - @PutExchange para requisições HTTP PUT
 - @PatchExchange para requisições HTTP PATCH
 - @DelectExchange para requisições HTTP DELETE

HTTP Interface

- Suponha que você precisa inovar as rotas deste controller:

```
@RestController
@RequestMapping("ProdutoReativo")
public class ProdutoController {

    @Autowired
    private ProdutoService service;

    @GetMapping("all")
    public Flux<ProdutoDto> all(){
        return this.service.getAll();
    }

    @GetMapping("price-range")
    public Flux<ProdutoDto> getByPriceRange(@RequestParam("min") int min,
                                           @RequestParam("max") int max){
        return this.service.getProductByPriceRange(min, max);
    }

    @GetMapping("{id}")
    public Mono<ResponseEntity<ProdutoDto>> getProductById(@PathVariable String id){
        return this.service.getProductById(id)
            .map(ResponseEntity::ok)
            .defaultIfEmpty(ResponseEntity.notFound().build());
    }

    @PostMapping
    public Mono<ProdutoDto> insertProduct(@RequestBody Mono<ProdutoDto> productDtoMono){
        return this.service.insertProduct(productDtoMono);
    }
}
```

HTTP Interface

- Você precisa criar uma interface desta forma:

```
public interface ProdutoService {  
    @GetExchange("/ProdutoReativo/all")  
    Flux<ProdutoDto> getAll();  
  
    @GetExchange("/ProdutoReativo/price-range")  
    Flux<ProdutoDto> getByPriceRange(@RequestParam("min") int min, @RequestParam("max") int max);  
  
    @GetExchange("/ProdutoReativo/{id}")  
    Mono<ResponseEntity<ProdutoDto>> getProductById(@PathVariable String id);  
  
    @PostExchange("/ProdutoReativo")  
    Mono<ProdutoDto> insertProduct(@RequestBody Mono<ProdutoDto> productDtoMono);  
}
```

HTTP Interface

- Você precisa criar uma interface desta forma:

```
@Bean
public WebClient webClient() {
    return WebClient.create("http://localhost:8080");
}
```

```
HttpServiceProxyFactory httpServiceProxyFactory = HttpServiceProxyFactory
    .builder(WebClientAdapter.forClient(webClient))
    .build();
ProdutoService booksService = httpServiceProxyFactory.createClient(ProdutoService.class);

public Mono<ResponseEntity<ProdutoDto>> getProductById(String id) {
    return booksService.getProductById(id);
}
```